PackedSequence#

https://pytorch.org/docs/stable/generated/torch.nn.utils.rnn.PackedSequence.

Description:

Holds the data and list of batch_sizes of a packed sequence. All RNN modules accept packed sequences as inputs. Instances of this class should never be created manually. They are meant to be instantiated by functions like pack_padded_sequence(). Batch sizes represent the number elements at each sequence step in the batch, not the varying sequence lengths passed to pack_padded_sequence(). For instance, given data abc and x the PackedSequence would contain data axbc with batch_sizes=[2,1,1].

Function Signature

```
class torch.nn.utils.rnn.PackedSequence(data,
batch_sizes=None, sorted_indices=None,
unsorted_indices=None)[source]#

Variables

Return type

batch_sizes: Tensor#

count(value, /)#
```

Returns:

bool

Notes & Warnings

NOTE Note Instances of this class should never be created manually. They are meant to be instantiated by functions like pack_padded_sequence(). Batch sizes represent the number elements at each sequence step in the batch, not the varying sequence lengths passed to pack_padded_sequence(). For instance, given data abc and x the PackedSequence would contain data axbc with batch_sizes=[2,1,1].

NOTE Note data can be on arbitrary device and of arbitrary dtype. sorted_indices and unsorted_indices must be torch.int64 tensors on the same device as data. However, batch_sizes should always be a CPU torch.int64 tensor. This invariant is maintained throughout PackedSequence class, and all functions that construct a PackedSequence in PyTorch (i.e., they only pass in tensors conforming to this constraint).

NOTE Note If the self.data Tensor already has the correct torch.dtype and torch.device, then self is returned. Otherwise, returns a copy with the desired configuration.

Detailed Documentation

Documentation

Holds the data and list of batch_sizes of a packed sequence.

All RNN modules accept packed sequences as inputs.

Instances of this class should never be created manually. They are meant to be instantiated by functions like pack_padded_sequence().

Batch sizes represent the number elements at each sequence step in the batch, not the varying sequence lengths passed to pack_padded_sequence(). For instance, given data abc and x the PackedSequence would contain data axbc with batch_sizes=[2,1,1].

data (Tensor) – Tensor containing packed sequence

batch_sizes (Tensor) – Tensor of integers holding information about the batch size at each sequence step

sorted_indices (Tensor, optional) – Tensor of integers holding how this PackedSequence is constructed from sequences.

unsorted_indices (Tensor, optional) – Tensor of integers holding how this to recover the original sequences with correct order.

data can be on arbitrary device and of arbitrary dtype. sorted_indices and unsorted_indices must be torch.int64 tensors on the same device as data.

However, batch_sizes should always be a CPU torch.int64 tensor.

This invariant is maintained throughout PackedSequence class, and all functions that construct a PackedSequence in PyTorch (i.e., they only pass in tensors conforming to this constraint).

Alias for field number 1

Return number of occurrences of value.

Alias for field number 0

Return first index of value.

Raises ValueError if the value is not present.

Return true if self.data stored on a gpu.

Return true if self.data stored on in pinned memory.

Alias for field number 2

Perform dtype and/or device conversion on self.data.

It has similar signature as torch.Tensor.to(), except optional arguments like non_blocking and copy should be passed as kwargs, not args, or they will not apply to the index tensors.

If the self.data Tensor already has the correct torch.dtype and torch.device, then self is returned. Otherwise, returns a copy with the desired configuration.

Alias for field number 3